

Model strategy: which models, in what order, and why

Accepted-loan default prediction · written after Preprocessing V2, before any model is trained

The question every model answers

For a NEW loan application, what is the probability that the borrower will default (Charged Off / Default) instead of paying the loan back in full? The model outputs one number, the **probability of default (PD)**. Everything else the website shows (risk band, approve / review / decline, reasons) is built on top of that number by a separate **decision layer**; see the last two pages.

The plan in one table

We climb a ladder: each model is tested only after the previous one, and each step has a specific question it answers. A more complex model has to **earn** its place by beating the simpler one on the validation set by a meaningful margin.

#	Model	Question it answers	Role
M0	Dummy baseline (always predicts the training default rate)	What score do we get with zero intelligence? Every real model must beat it.	Sanity floor
M1	Logistic Regression	How far does a simple, fully explainable model get? Is there leakage?	Baseline + explainable benchmark
M2	Random Forest	Do non-linear effects and feature interactions add real signal over M1?	Non-linearity check
M3	LightGBM (gradient boosting)	What is the best accuracy we can reach on this data?	Main candidate
M4	XGBoost (gradient boosting)	Is M3's result real, or specific to one library / setting?	Confirmation / challenger
M5	Scorecard (binned Logistic Regression)	Can we express the model as a classic credit 'points' scorecard?	Optional, for interviews
	Calibration + decision layer (after picking a model)	Are the probabilities honest? Where do approve / review / decline thresholds go?	Turns PD into decisions

Every model is trained twice: version A and version B

A uses all 190 features. **B** removes the 42 LendingClub-derived columns (grade, sub_grade, int_rate, installment). Page 4 explains why this comparison is the most important experiment in the project.

1. What the preprocessed data tells us (and how it shapes the plan)

Model choice should follow from the data, not from habit. These are the facts from Preprocessing V2 and what each one means for modeling:

Fact from preprocessing	What it means for modeling
Train 388,124 rows, validation 162,745, test 212,801; 190 numeric features.	Plenty of data for gradient boosting; Logistic Regression trains in seconds. No need for deep learning.
Default rate: train 17.24%, validation 20.35%, test 20.06%.	Classes are imbalanced (about 1 bad in 5-6). Accuracy is useless : predicting 'everyone repays' scores 83% accuracy and catches zero defaults. We use ranking and probability metrics instead (page 3).
The default rate rises from 17% to 20% between train and validation/test.	This is real drift over time. Rankings usually survive drift; raw probabilities may be too low, so we calibrate the final model on the validation set.
Strongest single feature: int_rate with AUC 0.674; every other feature is weaker.	No single variable decides default. Signal comes from combining many weak features, which is where tree ensembles (M2-M4) usually beat a linear model. It also means there is no obvious leakage.
Features are unscaled; money columns (income, balances, limits) are heavily skewed.	Logistic Regression needs scaling (and log transforms help); tree models do not care about scale or skew.
18 'was missing' flags; missing often means 'never happened' (e.g. months since last delinquency).	Trees use these flags naturally; Logistic Regression gets them as ordinary 0/1 inputs.
fico_range_low and fico_range_high are almost identical (high = low + 4).	Harmless for trees; for Logistic Regression keep one, or coefficients become unstable.
zip_code, emp_title, title are target-encoded (smoothed default rates, out-of-fold on train).	Usable directly by every model. zip_code can act as a proxy for protected groups, so we also test without it.
Splits are by time: train 2012-08 → 2014-12, validation 2015 H1, test 2015 H2; stress set 2016-2018 (518,744 rows).	All tuning uses validation. Test is used once , at the end. The stress set is only a 'what if' check, because its labels are biased (many loans were still running).

2. How we will judge every model

Every model gets the same report card, computed on the **validation** set while we are choosing, and on the **test** set once at the very end.

Metric	What it measures	Why we use it here
ROC-AUC (main metric)	Probability that a random defaulter gets a higher PD than a random good borrower.	Measures how well the model ranks risk, which is exactly what an approval engine needs; not fooled by imbalance.
PR-AUC	Precision vs recall for the default class.	Focuses on the minority class (defaults); complements ROC-AUC.
KS statistic	Largest gap between the score distributions of good and bad borrowers.	Industry-standard metric in credit risk; interviewers and banks expect it.
Brier score + calibration curve	Do predicted probabilities match reality? (PD 10% → about 10% default)	The decision layer uses PD as a real probability (expected loss, pricing), so it must be honest.
Approval-rate vs bad-rate curve	If we approve the safest X% of applicants, what share of them defaults?	Translates model quality into business language: 'approve 70% of applicants at a 12% bad rate'.
Segment checks	All metrics split by term (36 vs 60 months) and by year-half.	The 60-month loans in validation/test still carry some unfinished-loan bias, so we report them separately.

Ground rules

- **Validation chooses, test confirms.** Every decision (features, hyperparameters, thresholds) is made on validation. The test set is opened exactly once; if we peek and adjust, it stops being an honest estimate.
- **Simplest model wins ties.** If a complex model beats a simpler one by less than about 0.005 AUC, we keep the simpler one: it is easier to explain, debug and deploy.
- **Suspiciously good means suspicious.** Published results on LendingClub application data usually land around AUC 0.68-0.73. A score far above that (say 0.80+) means something is leaking, not that we are geniuses.
- **Same preprocessing for everyone.** All models read the same X/y Parquet files from `final_preprocessed_data_v2/`. Model-specific steps (scaling for Logistic Regression) live inside each model's own pipeline.
- **Imbalance is handled with weights, not fake data.** We compare class weights vs none. We skip SMOTE: with 66,928 real training defaults there is no shortage of examples, and synthetic loans distort probabilities.

3. The ladder, step by step

M0 Dummy baseline	
What it is	A 'model' that gives every applicant the same PD: the training default rate (17.24%).
Why run it	It costs nothing and sets the floor: ROC-AUC 0.50 and a reference Brier score. If a real model barely beats it, something is wrong with the pipeline.

M1 Logistic Regression · the explainable baseline	
Why first	It is the standard model in real credit scoring: every coefficient has a meaning ('each extra point of DTI raises the odds of default by x%'), regulators accept it, and it trains in seconds. It sets the bar every complex model must clear.
What we learn	(1) A realistic baseline AUC. (2) A leakage check: if a linear model scores far above 0.75 the data is leaking. (3) Which features push risk up or down, as a sanity check against credit intuition (higher FICO → lower risk, higher DTI → higher risk).
Setup	Pipeline inside the model: log-transform skewed money columns, StandardScaler, L2 regularisation; drop fico_range_high; tune C on validation; try class_weight=None vs 'balanced'.
Watch out	It can only draw straight-line relationships, so it misses interactions such as 'high DTI is only dangerous when income is low'. That gap is exactly what M2-M4 test.
Success looks like	Validation AUC clearly above 0.5 and in the normal range for this data; coefficients with sensible signs; a calibration curve close to the diagonal.

The most important experiment: A (with LendingClub grades) vs B (without)

int_rate, installment, grade and sub_grade are **LendingClub's own risk assessment**. int_rate is our single strongest feature (AUC 0.674). Model A will almost certainly score higher. But ask: **when our engine scores an application, does that information exist yet?**

- **If we are the lender deciding approve / decline:** grade and interest rate are set *after* the risk decision, using the risk decision. Our engine cannot use them, so **Model B is the deployable model**. Our own PD can then be used to set the grade and price.
- **If we were an investor picking loans already graded by LendingClub:** Model A would be legitimate.

Plan: train both, report the gap (A minus B), **deploy B**, and use A as a benchmark that shows how close our independent model gets to LendingClub's own underwriting.

M2 Random Forest · does non-linearity matter?

Why second	It is the simplest strong non-linear model: hundreds of decision trees on random subsets of rows and features, averaged. It needs almost no tuning and no scaling, and captures interactions automatically.
What we learn	The size of the gap between M1 and M2 tells us how much signal is non-linear. A big gap → boosting (M3/M4) is worth the effort. No gap → the data is mostly linear and Logistic Regression may be enough.
Setup	300-500 trees, limited depth / minimum leaf size, class_weight balanced_subsample vs none. Run on all 388,124 training rows (or a sample for speed during exploration).
Watch out	Slow and memory-hungry on 388k × 190 with 16 GB RAM; probabilities are usually poorly calibrated (squeezed toward the middle), so calibration is mandatory if it were chosen.
Success looks like	A clear AUC gain over M1 on validation, consistent across 36- and 60-month loans.

M3 LightGBM · the main candidate

Why	Gradient boosting builds trees one after another, each fixing the previous trees' mistakes. On tabular data like ours it is usually the most accurate family. LightGBM is very fast on hundreds of thousands of rows and is widely used for credit scoring in industry.
What we learn	The realistic accuracy ceiling for this dataset, and which features drive risk (via SHAP values).
Setup	Early stopping on validation (stop adding trees when validation AUC stops improving); tune learning rate, number of leaves, minimum data per leaf, feature / row subsampling, L1/L2 penalties; a small, fixed tuning budget (e.g. Optuna, 30-50 trials) so we do not overfit the validation set.
Bonus	Supports monotonic constraints : we can force 'higher FICO never increases risk' and 'higher DTI never decreases risk'. That makes the model safer and easier to defend, usually at a tiny accuracy cost.
Watch out	Easy to overfit with too many trees or leaves; probabilities need checking; less transparent than M1, so we pair it with SHAP explanations.
Success looks like	Best validation AUC / KS of the ladder by a meaningful margin over M1 and M2, stable across segments, sensible SHAP patterns.

M4 XGBoost · the challenger

Why	Same family as LightGBM but a different tree-growing strategy and implementation. If both land on a similar score, we can trust the result; if they disagree a lot, something in our setup needs a closer look.
Setup	hist tree method, early stopping on validation, a similar tuning budget to M3, scale_pos_weight vs none.
Decision	Keep whichever of M3 / M4 is better on validation; if within about 0.005 AUC, keep the faster and simpler one.

M5 Scorecard (optional) · binned Logistic Regression

What it is	The classic bank 'points' scorecard: each feature is cut into bins, each bin gets points via Weight of Evidence, and the points add up to a credit score (e.g. 600 + 20 points per doubling of odds).
Why consider it	It is what many real lenders still deploy, it is extremely explainable, and building one is an excellent interview talking point. It is optional because it rarely beats boosting on accuracy.

Models we deliberately skip, and why

Model	Why we skip it
Single decision tree	Unstable (small data changes give a different tree) and clearly weaker than forests and boosting. Useful only as a teaching picture.
k-Nearest Neighbours, SVM	Very slow on 388k rows × 190 features and no accuracy advantage on tabular data; probabilities are awkward.
Naive Bayes	Assumes features are independent; ours are strongly correlated (balances, limits, counts).
Neural networks (MLP)	On medium-sized tabular data they rarely beat gradient boosting, need much more tuning, and are harder to explain to a credit committee.
CatBoost	Its big advantage is native handling of raw categorical columns, but our categories are already encoded. Could be added later as an extra challenger.
SMOTE / oversampling	We have 66,928 real defaults in training; class weights are enough and do not invent fake loans or distort probabilities.

4. After the ladder: from the best model to a deployable engine

Step	What we do	Why
1. Pick the winner	Compare M0-M4 (A and B) on validation: AUC, KS, PR-AUC, Brier, segment stability. Apply the 'simplest model wins ties' rule. Expected winner: LightGBM or XGBoost, version B.	One honest, documented choice instead of 'the model that looked best on the test set'.
2. Calibrate	Fit Platt scaling or isotonic regression on validation predictions.	Corrects drift (17% → 20%) and class-weight effects so PD = 10% really means about 10% default.
3. Explain	SHAP values for the tree model: global feature importance plus the top reasons for each individual application.	Every decline must come with reasons (US lending rules require 'adverse action' reasons). Also a final leakage sanity check.
4. Fairness check	Compare approval and error rates across states / zip regions; re-run the winner without zip_code and measure what it costs.	zip codes can act as a proxy for race or ethnicity (fair-lending risk).
5. Set the decision layer	Choose PD thresholds and risk bands on validation using a business target, e.g. 'approve as many applicants as possible while keeping the approved bad rate under X%'.	Thresholds are a business decision, not a model output; they can change without retraining.
6. Final test (once)	Score the untouched test set; report every metric and the approval-rate vs bad-rate curve.	The single honest estimate of real-world performance, the number for the resume and README.
7. Stress check	Score the 2016-2018 stress set and compare the ranking quality (AUC).	Shows how the model behaves on newer loans; read with care because those labels are biased.
8. Save and serve	Save the calibrated model next to preprocessor_v2.joblib with a version number; build the API.	Leads straight into the website: see the next two pages.

Where this fits in the whole project

- **Part 1 (this plan), accepted loans:** predicts the probability that an approved-type applicant defaults.
- **Part 2, rejected loans (later):** the rejected dataset only has a few columns (amount, title, risk score, DTI, zip, state, employment length). It can power a first-stage screening model ('would LendingClub have accepted this?') and helps discuss **reject inference**: our default model only ever saw approved borrowers.
- **Possible Part 3, loss given default:** recoveries were rightly removed from the PD model (they happen after default), but on defaulted loans they can train a separate LGD model, so that expected loss = PD × LGD × exposure.

5. Deployment: what goes IN

On the website the user fills in a form; the backend adds credit-bureau data and calls the scoring API. Preprocessing V2 actually uses **73 raw input fields** (of the 103 it was given; the rest were dropped as empty in training). They come from four places:

Source	Fields	Count
Applicant types it in the form	application date, loan_amnt, term, purpose, title, emp_title, emp_length, home_ownership, annual_inc, zip_code, addr_state	11
Lender computes it	verification_status (was income verified?), dti (monthly debt ÷ monthly income)	2
Credit bureau report (pulled by the backend)	FICO range, earliest credit line, open / total accounts, revolving balance and utilisation, delinquencies, inquiries, public records, bankruptcies, card limits, months-since fields, ...	56
LendingClub pricing (Model A only)	int_rate, installment, grade, sub_grade: not needed by the deployed Model B	4

Example request (illustrative values):

```
POST /api/v1/score
{
  "application_date": "2026-10",          # becomes issue_d -> credit history length
  "applicant": {
    "loan_amnt": 12000,                    "term": "36 months",
    "purpose": "debt_consolidation",      "title": "Debt consolidation",
    "emp_title": "Data Analyst",           "emp_length": "3 years",
    "home_ownership": "RENT",              "annual_inc": 72000,
    "zip_code": "940xx",                   "addr_state": "CA"
  },
  "lender_computed": { "verification_status": "Verified", "dti": 18.4 },
  "credit_bureau": {
    "fico_range_low": 700, "fico_range_high": 704, "earliest_cr_line": "Mar-2010",
    "open_acc": 9, "total_acc": 20, "revol_bal": 8500, "revol_util": 41.2,
    "delinq_2yrs": 0, "inq_last_6mths": 1, "pub_rec": 0, "mort_acc": 0,
    "...": "45 more bureau fields (missing ones are allowed: they are imputed)"
  }
}
```

How the request flows through the system

Website form	→ API endpoint (e.g. FastAPI /score)	→ preprocessor_v2.joblib 103 raw → 190 features	→ model.joblib 190 features → PD	→ Decision layer PD → band, decision, reasons	→ JSON response shown on website
---------------------	---	--	---	---	--

For a demo website without a real credit bureau, the bureau block can be typed in manually, pre-filled from a sample profile, or left partly empty (the pipeline fills gaps with training medians and 'was missing' flags).

6. Deployment: what comes OUT (sample response)

Important distinction: the trained model returns exactly **one number**, the probability of default. It does **not** classify individual variables. The class label, risk band, decision, pricing and reasons are added by the **decision layer** (thresholds and policy rules we choose on the validation set) and by SHAP explanations.

```
{
  "application_id": "APP-2026-10-000123",
  "model_version": "pd_lightgbm_B_v1 (calibrated)",
  "scored_at": "2026-10-05T15:20:11Z",

  "probability_of_default": 0.083,          # <- the ONLY thing the model itself returns
  "risk_band": "B",                        # decision layer: PD 5-10% -> band B
  "predicted_class": "repay",              # decision layer: PD below the 0.20 cut-off -> "repay"
  "decision": "APPROVE",                   # APPROVE / MANUAL_REVIEW / DECLINE
  "decision_reasons": ["PD 8.3% below approval threshold of 10%",
                       "All hard policy rules passed (DTI <= 40, no bankruptcy in 12 months)"],

  "expected_loss": {"exposure": 12000, "lgd_assumed": 0.85, "amount": 846.60},
  "suggested_pricing_tier": "B2",          # optional: risk-based pricing from PD

  "top_risk_factors": [                    # SHAP reason codes for THIS applicant
    {"feature": "revol_util", "value": "41.2%", "effect": "+1.1 pts PD",
     "text": "Credit cards fairly utilised"},
    {"feature": "purpose", "value": "debt_consolidation", "effect": "+0.6 pts PD",
     "text": "Debt consolidation loan"}
  ],
  "top_protective_factors": [
    {"feature": "term_months", "value": "36", "effect": "-2.4 pts PD",
     "text": "Short 36-month term"},
    {"feature": "annual_inc", "value": "$72,000", "effect": "-1.3 pts PD",
     "text": "Income comfortably covers payments"}
  ],
  "warnings": ["3 bureau fields missing -> imputed with training medians"]
}
```

Field	Comes from	Meaning
probability_of_default	Model (calibrated)	Chance this loan ends Charged Off / Default. 0.083 = 8.3%.
risk_band	Decision layer	PD bucket, e.g. A <5%, B 5-10%, C 10-17%, D 17-25%, E 25-35%, F >35% (set on validation).
predicted_class	Decision layer	'default' / 'repay' using one cut-off. Only a convenience label; PD carries the information.
decision	Decision layer	e.g. PD <10% approve; 10-20% manual review; >20% decline; plus hard rules that override the model.
expected_loss	Decision layer	PD × LGD × exposure. LGD is a placeholder until we model recoveries (Part 3).
top_*_factors	SHAP on the model	Why this applicant got this PD; needed for decline letters and for user trust.

All numbers on this page are **illustrative** (and the # comments are only explanations, not part of the real JSON): no model has been trained yet. Thresholds, bands and LGD will be set from validation results; the response format is what the website will receive.